

Trabajo Práctico Especial - RP-Lang

Instituto Tecnológico de Buenos Aires - Autómatas, Teoría de Lenguajes y
Compiladores (72.39)

Grupo RP-Lang

Ignacio Searles
isearles@itba.edu.ar
64.536

Augusto Barthelemy Solá
abarthelemysola@itba.edu.ar
64.502

Santiago Bassi
sabassi@itba.edu.ar
64.643

21 de junio de 2025

Resumen

El presente informe trata sobre el diseño e implementación de un lenguaje que permite la definición y evaluación de funciones recursivas primitivas. El compilador de **RP-Lang** traduce programas escritos en este lenguaje a código C, aprovechando la expresividad y eficiencia de dicho lenguaje de bajo nivel para ejecutar las funciones definidas.

Contents

1	Introducción	1
2	Modelo computacional	2
2.1	Dominio	2
2.2	Lenguaje	2
3	Implementación	3
3.1	Frontend	3
3.2	Backend	4
3.3	Dificultades encontradas	5
4	Futuras Extensiones	6
5	Conclusiones	6
6	Referencias	6
7	Bibliografía	6

1 Introducción

Las funciones recursivas primitivas constituyen una clase fundamental de funciones total computables, ampliamente estudiadas en teoría de la computación. En este trabajo se presenta **RP-Lang**, un lenguaje de programación diseñado para expresar y evaluar exclusivamente este tipo de funciones.

El desarrollo del lenguaje incluye el diseño de su sintaxis y semántica, así como la implementación de un compilador que traduce programas **RP-Lang** a código en lenguaje C. Para ello, se utilizaron las herramientas Flex y Bison, facilitando la construcción de los analizadores léxico y sintáctico respectivamente.

El objetivo principal del proyecto es proporcionar una herramienta que permita experimentar con funciones recursivas primitivas de forma práctica.

2 Modelo computacional

2.1 Dominio

Desarrollar un lenguaje que permita definir funciones recursivas primitivas (RP). Las funciones RP son de la forma $f : \mathbb{N}^k \rightarrow \mathbb{N}$ (Zahnd, 2022). Las mismas cumplen que están definidas en base a los esquemas recursivos de tipo I y tipo II y composición a partir de las funciones iniciales:

- $suc : \mathbb{N} \rightarrow \mathbb{N}, suc(x) = x + 1$
- $const_k : \mathbb{N} \rightarrow \mathbb{N}, const_k(x) = k$
- $\pi_j^k : \mathbb{N}^k \rightarrow \mathbb{N}, \pi_j^k(x_1, \dots, x_k) = x_j$

Las definiciones de funciones RP pueden ser composiciones o esquemas recursivos:

- Para la composición, si h y g son RP $\implies f(x) = g(h(x))$ es RP.
- Para el esquema recursivo de tipo I, si $g : \mathbb{N}^2 \rightarrow \mathbb{N}$ es RP $\implies f : \mathbb{N} \rightarrow \mathbb{N}$ tal que $f(0) = k \in \mathbb{N}$ y $f(n+1) = g(n, f(n))$ es RP.
- Para el esquema recursivo de tipo II, $g : \mathbb{N}^{n+2} \rightarrow \mathbb{N}$ y $q : \mathbb{N}^n \rightarrow \mathbb{N}$ son RP $\implies f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ tal que $f(x_1, \dots, x_n, 0) = q(x_1, \dots, x_n)$ y $f(x_1, \dots, x_n, y+1) = g(x_1, \dots, x_n, y, f(x_1, \dots, x_n, y))$ es RP.

El lenguaje permite evaluar cualquier función RP que se haya definido. Haciendo uso de las definiciones de las mismas para evaluarlas recursivamente y obtener un valor de salida.

2.2 Lenguaje

Un programa de **RP-Lang** (extensión recomendada **.rp**), está compuesto por definiciones, evaluaciones e **#include**. Los mismos puede aparecer intercalados en cualquier orden, mientras se cumpla que no se utilicen funciones no definidas en una definición. A continuación se listan construcciones del lenguaje:

- Se pueden definir funciones partiendo de las funciones básicas.
- Se pueden definir funciones mediante composición de funciones.
- Se pueden definir funciones mediante esquemas recursivos de tipo I y II.
- Se pueden evaluar funciones utilizando constantes o entradas por teclado.
- Se pueden importar funciones desde otros archivos.
- Las funciones binarias se pueden evaluar mediante la syntaxis: **(expr1 func expr2)** donde $f : \mathbb{N}^2 \rightarrow \mathbb{N}$.
- Se pueden incluir comentarios en el código.

Las funciones pueden definirse mediante expresiones compuestas o mediante esquemas recursivos. A continuación se muestran ejemplos representativos de cada caso.

Composición

```
#include times.rp

def pow3(x):
=> pow3(x) = (x * (x * x))

//this will prompt the user for input
pow3(input)
```

Recursión (esquema tipo I)

```
#include times.rp

def !(n):
-> !(0) = 1
-> !(n + 1) = *(suc(n), !(n))

!(4)
```

Recursión (esquema tipo II)

```
#include times.rp

def ^(x, y):
-> ^(x, 0) = 1
-> ^(x, y + 1) = *(x, ^(x, y))

(2 ^ 5)
```

3 Implementación

3.1 Frontend

Para la implementación del frontend se utilizaron las herramientas:

- Flex para el análisis léxico.
- Bison para el análisis sintáctico.

Durante esta etapa, el lenguaje sufrió leves modificaciones con respecto a su diseño original con el fin de resolver ambigüedades presentes en la gramática y facilitar el análisis sintáctico.

El frontend incorpora diversas validaciones estructurales. Sin embargo, otras validaciones semánticas más específicas, como correcta cantidad de argumentos, orden correcto de argumentos, entre otros, se delegan al backend del compilador para una mayor flexibilidad y separación de responsabilidades.

Como resultado de esta fase, se genera un árbol de sintaxis abstracta (AST) que representa la estructura lógica del programa y sirve como base para las etapas posteriores del proceso de compilación.

3.2 Backend

El backend del compilador toma como entrada el árbol de sintaxis abstracta (AST) generado por el frontend y produce como salida el código equivalente en lenguaje C. La generación de código se realiza en una única pasada recursiva sobre el AST, asistida por una tabla de funciones que contiene información relevante sobre cada función definida en el programa.

Esta tabla de funciones actúa como contexto y almacena, para cada función, su nombre original, nombre traducido al C (dado que **RP-Lang** permite nombres no alfanuméricos), cantidad de argumentos y lista de parámetros. Para su implementación se utilizó una estructura de tipo hashmap, provista por la librería por **Joshua J Baker** (Baker, 2020), donde la clave es el identificador de la función en **RP-Lang** y el valor es una estructura con los metadatos necesarios para la generación de código.

Durante el recorrido del AST, el backend realiza varias validaciones semánticas que no eran posibles o convenientes de implementar en el frontend. Estas validaciones incluyen:

- Validar cantidad correcta de parametros al momento de evaluar y definir una función.
- Validar orden correcto de los parametros en el caso base y chequeo de último parametro del caso base igual a la constante 0.
- Validar orden correcto de los parametros en el paso recursivo.
- Validar que en las definiciones los símbolos utilizados esten definidos en el scope de la función.
- Validar que en los factores en las definiciones no puedan ser entradas por teclado.
- Validar que las funciones utilizadas esten definidas previamente.
- Validar que al definir una función la misma no este definida previamente.

Una vez que todas las validaciones han sido superadas, se genera el código C correspondiente. El código producido mantiene la estructura funcional del programa original, pero expresada mediante funciones recursivas en C.

A continuación se muestra la estructura del código C generado por el compilador de **RP-Lang** dado un programa válido de entrada.

```
#include<stdio.h>

int get_positive_integer_from_stdin(){
int out = 0; scanf("%d", &out);
return (out < 0 ? 0 : out);
}

int suc(int n){
return n+1;
}

// definitions go here

// example definition
int fun1(int x, int y){
if (y == 0) {
return x ;
}
}
```

```

y--;

return suc(fun1( x , y ));
}

int main() {
//evaluations go here

//example evaluation. Evaluations are printed
printf("%d", fun1( 5 , 2 ));

return 0;
}

```

Cabe destacar que, debido a la naturaleza de las funciones recursivas primitivas, el código generado no es eficiente desde el punto de vista computacional. Por ejemplo, para calcular $x + y$ se requiere realizar y llamadas recursivas, replicando la definición formal de la suma como función RP. Esto va a ser general para todas las funciones RP, si $f : \mathbb{N}^k \rightarrow \mathbb{N}$ tal que $f(x_1, \dots, x_k)$, requiere realizar x_k llamadas recursivas. Esta ineficiencia es inherente al modelo teórico que el lenguaje busca representar. Aunque algunas de estas funciones podrían implementarse de forma iterativa para mejorar el rendimiento, ese no es el objetivo de **RP-Lang**, que prioriza la fidelidad al modelo matemático por sobre la eficiencia.

3.3 Dificultades encontradas

A lo largo del desarrollo del compilador se presentaron diversas dificultades técnicas. A continuación se detallan algunas de las más relevantes:

- Ambigüedad entre definiciones recursivas y por composición: En el diseño inicial de la gramática, tanto las definiciones por composición como los casos base de definiciones recursivas compartían una misma regla gramatical del estilo: `DOUBLE_ARROW ID OPEN_PARENTHESIS arguments CLOSE_PARENTHESIS EQUALS expression`. Esta similitud generaba un conflicto de tipo shift/reduce al momento del análisis sintáctico con Bison. Para resolver esta ambigüedad, se introdujo una distinción explícita entre ambos tipos de definición: se optó por utilizar el símbolo `=>` (`DOUBLE_ARROW`) exclusivamente para definiciones por composición (ver Código 2.2), y el símbolo `->` (`SINGLE_ARROW`) para las ramas de definiciones recursivas (ver Código 2.2). Esta decisión simplificó la gramática y eliminó los conflictos durante el parsing.
- Reestructuración del AST para facilitar la generación de código: Durante la implementación del backend, al recorrer el árbol de sintaxis abstracta (AST), se identificó que ciertos nodos generados por el frontend no aportaban información útil para la generación de código o bien duplicaban estructuras. Esto dificultaba innecesariamente el recorrido del árbol y la emisión de código en C. En consecuencia, se realizaron ajustes en la estructura del AST desde el frontend, eliminando nodos redundantes y simplificando la representación interna del programa. Esta reestructuración no solo facilitó la implementación del backend, sino que también mejoró la claridad y mantenibilidad del código del compilador.

4 Futuras Extensiones

5 Conclusiones

6 Referencias

A continuación, se enumeran las fuentes citadas en el informe.

- Baker, J. J. (2020). GitHub - tidwall/hashmap.c: Hash map implementation in C. GitHub.
- Zahnd, M. (2022). GitHub - mzahnd/apuntes-9335-logica-computacional: Apuntes de la materia Lógica Computacional (93.35) del ITBA. GitHub.

7 Bibliografía

A continuación, se enumeran las fuentes consultadas para la elaboración de este trabajo.

- Wikipedia contributors. (2025, June 10). Primitive recursive function. Wikipedia.
- Lexical Analysis with Flex, for Flex 2.6.2: Patterns. (n.d.).